Splode.io

System Architecture & Technical Master Document

Lead Architect: Jagadeeswaran M.

August 2026

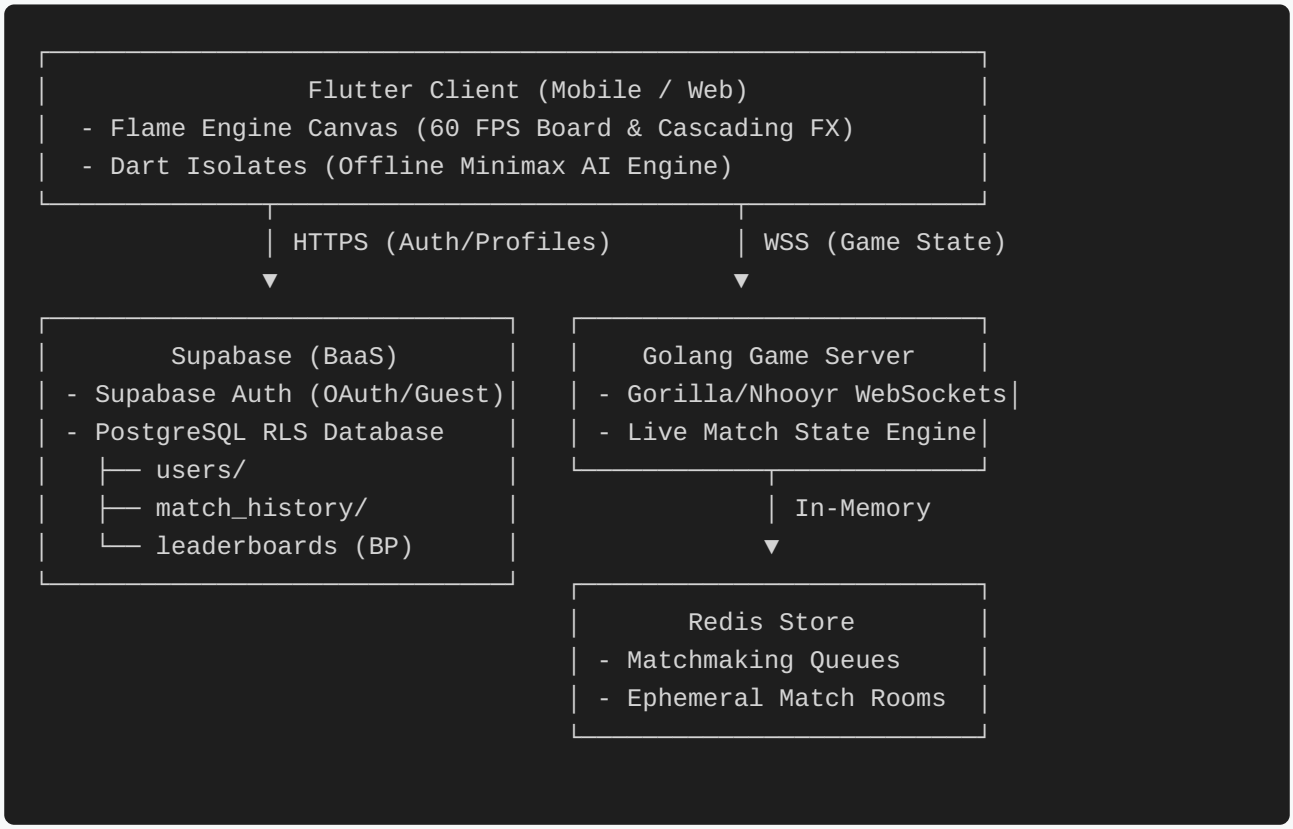
1. High-Level Concept & Brand Identity

- **Game Name: Splode.io** — A modern, esports-grade cross-platform competitive adaptation of the classic strategic board game *Chain Reaction*.
- **Rating System: Blast Points (BP)** — A competitive matchmaking ranking metric (analogous to standard MMR/Elo systems) that shifts dynamically based on match outcomes.

2. Core Game Rules & Mechanics

- **Grid Layouts:** Standard Mobile (9 × 6), Compact (6 × 6), and Large (12 × 8).
- **Cell Capacity (Critical Mass):**
  - *Corners:* Hold 1 orb (Explodes at 2).
  - *Edges:* Hold 2 orbs (Explodes at 3).
  - *Center:* Hold 3 orbs (Explodes at 4).
- **Turn & Explosion Rules:** Players alternate placing single orbs into empty cells or cells they own. Reaching critical mass triggers recursive orthogonal explosions, capturing adjacent opponent cells and cascading until the board stabilizes.
- **Win Condition:** Elimination occurs when a player loses all orbs after completing at least their first move. The last remaining player wins.

3. Comprehensive Technology Stack



Component Breakdown

- **Frontend Client:** Flutter + Flame Engine compiled natively for iOS, Android, and Web. Handles the UI layout, touch interactions, and high-performance 60 FPS particle rendering.
- **Offline AI Engine:** Dart Isolates running a Minimax Algorithm with Alpha-Beta Pruning on the mobile device, ensuring zero-latency solo play without burdening the server.
- **Real-Time Multiplayer Backend:** Golang (Go) utilizing lightweight Goroutines and high-throughput WebSockets to handle real-time move validation and state broadcasting for active 1v1 matches.
- **In-Memory Cache & Matchmaking:** Redis managing high-speed sorted sets (ZSET) for Blast Points (BP) matchmaking brackets and ephemeral match session tracking.
- **Meta-Game, Auth & Persistence:** Supabase (PostgreSQL + Supabase Auth) managing secure user logins (Google, Apple, Guest), relational profile data, historical match logs, and global Blast Points (BP) leaderboards backed by Row-Level Security (RLS).

4. Real-Time WebSocket Protocol (Client ↔ Golang Server)

All live multiplayer frames use lightweight JSON payloads sent across authenticated WebSocket connections.

Client-to-Server Events

- `JOIN_QUEUE`: `{ "mode": "ranked" | "casual", "gridSize": "9x6" }` — Enters player into the Redis BP queue.
- `LEAVE_QUEUE`: `{}` — Exits the active search queue.
- `MAKE_MOVE`: `{ "matchId": "uuid", "row": 2, "col": 3 }` — Submits an orb placement.
- `RESIGN_MATCH`: `{ "matchId": "uuid" }` — Forfeits the active game.

Server-to-Client Events

- `MATCH_FOUND`: `{ "matchId": "uuid", "players": [...], "grid": [9, 6], "currentTurn": "uid" }` — Triggers board initialization.
- `MOVE_ACCEPTED`: `{ "matchId": "uuid", "row": 2, "col": 3, "player": "uid", "nextTurn": "uid" }` — Authorizes local animation simulation.
- `MATCH_OVER`: `{ "winner": "uid", "bpDelta": { "uid1": +18, "uid2": -18 } }` — Ends the match and updates global Blast Points (BP).

5. Board Logic & AI Heuristics Specification

Explosion Cascade Algorithm (Pseudocode)

```
interface Cell {
  player: string | null;
  orbs: number;
}

func ProcessMove(board [][]Cell, row int, col int, playerId string)
[]ExplosionStep {
  // Increments target cell and executes recursive critical mass cascades...
}
```

Minimax Heuristic Evaluation Function

For an AI evaluating board state S for player P:

$$E(S) = w_1 \cdot C_{count} + w_2 \cdot O_{count} + w_3 \cdot U_{vulnerable} + w_4 \cdot K_{critical}$$

- C_{count}: Controlled cell advantage over the opponent.
- O_{count}: Net total orb count advantage.
- $U_{vulnerable}$: Penalty for friendly cells sitting adjacent to an opponent's critical cell.
- $K_{critical}$: Positional weighting rewarding stable corner/edge control and penalizing vulnerable center cells near critical mass.